API Documentation

Usage tricks

When you put down your controllers, the experience automatically goes into hand tracking mode.

Hold down both joystick buttons (or pinch both thumbs to pinky fingers) to transform the virtual world. Hold down just one joystick button (or pinch one thumb to pinky finger) to translate the virtual world.

Hold down both side triggers (or, if hand tracking, pinch thumb to middle finger with both hands) to teleport your controllers or hands across the room. The further apart your controllers or hands are from each other, the further away from you your virtual controllers or hands will go. To bring your controllers or hands back to you, do the same gesture while touching your two controllers or two forefingers together.

Coding basics

Scenes

All scenes are defined in `js/scenes/scenes.js`

In that file you will see lines like this:

```
{ name: "example1", path: "./example1.js", public: true },
```

Where `name` is the name that appears in the XR experience, `path` is the relative path to the scene file, and `public` defines whether you can see it directly in the scene menu after you enter AR mode.

To access non-public scenes in the XR experience: point to and hold any button in the scene menu with one controller, and click the same button with the other controller.

Each scene is a separately defined .js file located under `js/scenes`
The basic structure of a scene looks like this:

```
export const init = async model => {
   // initialization code, executed once on load
   model.animate(() => {
      // update code, executed every frame
   });
}
```

In short, `init` and `animate` are the most important functions in a scene.
You can refer to `js/scenes/simple.js` for a simple example.

Support libraries

cg

A 3D point or vector is represented as a an array of 3 floating point numbers:

`[x,y,z]`

A 4x4 matrix is represented as a column-major array of 16 floating point numbers:

`[x0,x1,x2,x3, y0,y1,y2,y3, z0,z1,z2,z3, w0,w1,w2,w3]`

SCALAR FUNCTIONS

```
cg.bezier(t,a,b,c,d)     // Scalar bezier interpolation (args are all floats)
cg.clamp(t,lo,hi)        // Clamps t to between lo and hi. lo defaults to 0, hi to 1
cg.def(value, default)   // If value is undefined, return default, else return value
cg.ease(t)               // Cubic ease curve: t<0 then 0, t>1 then 1, else t*t*(3-2*t)
cg.hermite(t,a,b,da,db) // Scalar hermite interpolation (args are all floats)
cg.ik(a,b,C,D)           // 2-link inverse kinematics:
                         // The shoulder is assumed to be at the origin
                         //    Input : a,b are limb lengths
                         //            C is a vector specifying wrist position
                         //            D is a vector suggesting elbow direction
                         //    Output: computed elbow position is placed in D
cg.mixAngle(a,b,t)       // Interpolate between a and b in radians with wrap-around
cg.mixf(a,b,t)           // Return a+t*(b-a), where a and b are floating point numbers
cg.mixf(a,b,t,u)         // Return a*t + b*u, where a and b are floating point numbers
cg.noise(x,y,z)          // Return value of noise at point [x,y,z] (Perlin1985)
cg.plateau(a,b,c,d,t)    // as t varies from a⟹b⟹c⟹d, return linear ramp 0⟹1⟹1⟹0
cg.roundFloat(n,f)       // Floating point value of f rounded to n digits
cg.uniqueID()            // Return a unique randomly defined integer ID
```

STRING FUNCTIONS

```
cg.decimal(f,n)          // String representation of f with exactly n decimal digits
cg.fixedWidth(f,n,d)     // String representation, n chars before dot, d digits after
cg.hexToRgba(hex)        // Convert hex representation of color to an [r,g,b,a] vector
cg.pack(array,lo,hi)     // Pack an array into a base 92*92 string; specify lo and hi
cg.round(f,n)            // String representation of f rounded to n digits
cg.unpack(string,lo,hi) // Unpack base 92*92 array; lo,hi must match packed values
```

VECTOR FUNCTIONS

```
cg.add(a,b)              // Return sum of two vectors
cg.cross(a,b)            // Return cross product of two vectors
cg.dot(a,b)              // Return dot product of two vectors
cg.mix(a,b,t)            // Return a+t*(b-a), where a and b are vectors
cg.mix(a,b,t,u)          // Return a*t + b*u, where a and b are vectors
cg.norm(v)               // Return norm (geometric length) of a vector
cg.normalize(v)          // Return v scaled to unit length
cg.roundVec(n,v)         // Return approximation of v, rounding components to n digits
cg.scale(v, s)           // Return v scaled by s
cg.subtract(a,b)         // Return difference of two vectors
```

CURVE FUNCTIONS

A curve is represented as an array of 3d points: `[ [x,y,z], [x,y,z], … ]`

```
cg.computerCurveLength(curve)    // Compute geometric curve length
cg.copyCurve(curve)              // Duplicate a curve
```

```
cg.resampleCurve(curve,n)        // Resample a curve to have n values
cg.sample(curve, t, isAngle)     // Sample a curve, 0<=t<=1, specify if in radians
cg.spline(keys,dst,ptsPerKey=10) // Create a spline. If dst, copy the result there
```

3D GEOMETRY METHODS

```
cg.isLineIntersectPoly(A,B,P) // Does A…B cross convex polyhedron bounded by planes P?
cg.isPointInBox(P,B)          // Is P inside a box? B is xform matrix of unit cube
cg.isRayIntersectTriangle(V,W,A,B,C) // Does ray from V to W intersect triangle A,B,C?
cg.isSphereIntersectBox(S,B)  // Does sphere S=x,y,z,r enter box B? B is cube xform.
```

MATRIX FUNCTIONS

Each matrix is represented as a column-major flat array of 16 floating point numbers

```
cg.mAimX(vec,vec2)    // Return matrix rotating X to vec; optionally rotate Y to vec2
cg.mAimY(vec,vec2)    // Return matrix rotating Y to vec; optionally rotate Z to vec2
cg.mAimZ(vec,vec2)    // Return matrix rotating Z to vec; optionally rotate X to vec2
cg.mFromQuaternion(q) // Return matrix corresponding to quaternion {q.x,q.y,q.z,q.w}
cg.mHitRect(beam,rect)// Returns true if a ray hits a rectangle. beam is xform matrix
                      // of ray [[0,0,0]=>[0,0,-1]], rect is xform of unit square
cg.mIdentity()        // Return identity matrix
cg.mInverse(m)        // Return inverse of matrix m
cg.mMirrorZ(m)        // Return matrix that flips z without turning object inside out
cg.mMultiply(a,b)     // Return product of two matrices
cg.mPerspective(fl)   // Return perspective transformation along Z
cg/mProject(x,y,z)    // Return projective matrix
cg.mRotateX(theta)    // Return matrix that rotates about X
cg.mRotateY(theta)    // Return matrix that rotates about Y
cg.mRotateZ(theta)    // Return matrix that rotates about Z
cg.mScale(s)          // Return matrix that scales uniformly by floating point s
cg.mScale(vec)        // Return matrix that scales by vec, where vec = [x,y,z]
cg.mScale(x,y,z)      // Return matrix that scales by [x,y,z]
cg.mToQuaterion(m)    // Return quaternion {q.x,q.y,q.z,q.w} corresponding to matrix m
cg.mTransform(m,p)    // Return transformation of point p by matrix m
cg.mTranslate(vec)    // Return matrix that translates by vec, where vec = [x,y,z]
cg.mTranslate(x,y,z)  // Return matrix that translates by [x,y,z]
cg.mTranspose(m)      // Return matrix transpose
```

MATRIX STACK OBJECT AND METHODS

- All methods other than `save` and `restore` operate on only the top matrix of the stack
- Methods that act on the top matrix of the stack are built on top of the equivalent `cg.*` functions
- All methods other than `getValue()` return the `matrix` object itself (so method calls can be chained)

```
let matrix = cg.Matrix() // Instantiate a 4x4 matrix stack object

matrix.aimX(vec)
matrix.aimY(vec)
matrix.aimZ(vec)
matrix.getValue()        // Return a flat array of 16 floating point numbers
matrix.identity()
matrix.inverse()
matrix.project(x,y,z)
```

```
matrix.restore()         // Restore matrix from the stack
matrix.rotateX(theta)
matrix.rotateY(theta)
matrix.rotateZ(theta)
matrix.save()             // Push top matrix onto the stack
matrix.scale(s)
matrix.scale(vec)
matrix.scale(x,y,z)
matrix.setValue(array)    // Argument is a flat array of 16 floating point numbers
matrix.translate(vec)
matrix.translate(x,y,z)
matrix.transpose()
```

PHYSICS OBJECTS

```
let deriv = new Deriv()   // Maintain the derivative of a changing value

deriv.get()
deriv.set(p)
deriv.update()

let spring = new Spring() // One-dimensional spring simulator

spring.getPosition()
this.setDamping(d)
this.setForce(f)
this.setMass(m)
this.update(deltaTime)

let dof = new DOF()       // One-dimensional damped oscillator simulator

this.setDamping(d)
this.setForce(f)
this.setMass(m)
this.setPosition(p)
this.update(deltaTime)
```

clay

- **API for the modeler implemented in** `js/render/core/clay.js`
- **The root node, called** `model`**, is the only object that is already created for you.**
- **You create a scene by building a tree hierarchy, using the** `obj.add(...)` **method.**

METHODS:

```
model.animate(() => PROCEDURE) // Called every animation frame
child = obj.add(type)
     // Valid types: 'sphere'|'tubeX'|'tubeY'|'tubeZ'|'cube'|'donut'
     //              or no argument to create an transformation node
     // Eg: let hip = pelvis.add();          // This is a transformation node
     //     let leg = hip.add('tubeY');      // This node will render a shape
child = obj.child(index)          // Return nth child of this node

obj = obj.aimX(vec)               // Aim tobject's X axis toward vec
```

```
obj = obj.aimY(vec)               // Aim object's Y axis toward vec
obj = obj.aimZ(vec)               // Aim object's Z axis toward vec
obj = obj.color([r,g,b] or r,g,b) // Set color: values are 0.0 … 1.0
obj = obj.identity()              // Set to identity matrix
obj = obj.info(info)              // Add extra shape info
obj = obj.move([x,y,z] or x,y,z)  // Translate by x,y,z
obj = obj.remove(child or index)  // Remove a child by ref. or index
obj = obj.scale([x,y,z] or x,y,z) // Non-uniform scaling
obj = obj.scale(s)                // Uniform scaling: s is floating pt
obj = obj.txtr(txtrUnit)          // Apply texture: txtrUnit is 0 … 14
obj = obj.turnX(radians)          // Rotate about the X axis
obj = obj.turnY(radians)          // Rotate about the Y axis
obj = obj.turnZ(radians)          // Rotate about the Z axis
```

VALUES:

```
model.time                        // Time in seconds since the program started
model.deltaTime                   // Time since previous animation frame
```

When any property is left unspecified for an object, that property is inherited from the object's parent. If the parent does not have that property, the search for a value continues all the way up to the root. All properties are defined by default for the root.

clientState

A global object that tells you the current input state of every client.
The implementation can be found in: `js/render/core/clientStateSharing.js`

Brief descriptions of clientState methods:

```
clientState.button(id,hand,b)    // Is controller button b pressed?
clientState.coords(id)           // Local coordinate system of client
clientState.finger(id,hand,i)    // xyz position of a finger tip
clientState.hand(id,hand)        // Matrix transform of client's hand
clientState.head(id)             // Matrix transform of client's head
clientState.isHand(id)           // Is client using hand tracking?
clientState.isXR(id)             // Is client in immersive mode?
clientState.pinch(id,hand,i)     // Pinch state of a finger
clientState.pinchState(id,hand,i)// Detailed pinch info for a finger
clientState.point(id,hand)       // [u,v] value if a hand is pointing
```

More detailed descriptions of clientState methods:

```
T_or_F = clientState.button(id,hand,b)
```

When not hand tracking, returns the press state of button `b`
When hand tracking, returns `null`

```
mat_16 = clientState.coords(id)
```

The current matrix transform of this client's local coordinate system

```
vec_3  = clientState.finger(id,hand,f)
```

When hand tracking, returns the 3D location of finger `f`
When not hand tracking, returns the 3D location of the controller's virtual ping pong ball

```
mat_16 = clientState.hand(id,hand)
```

When hand tracking, returns the matrix transform of the client's hand
When not hand tracking, returns the matrix transform of the controller

```
mat_16 = clientState.head(id)
```

The matrix transform of the client's head

```
T_or_F = clientState.isHand(id)
```

Is this client currently using hand tracking?

```
T_or_F = clientState.isXR(id)
```

Is this client currently in immersive mode?

```
T_or_F = clientState.pinch(id,hand,f)
```

When hand tracking, returns `true` if finger `f` is pinching, else the hand's pointing state.
When not hand tracking, returns the boolean press state of button `f-1`
`f` can be `1`,`2`,`3` or `4`, for your thumb touching your index, middle, ring or pinkie finger,
or else `5` for pointing with your thumb sideways or `6` for pointing with your thumb up.
If using a controller, `draw.pinch(hand)` returns true if the trigger is pressed.

```
state = clientState.pinchState(id,hand,f)
```

For when you need more detailed info about pinch state. Returns one of four states:

```
'press'    // State was previously up, and now it is down.
'down'     // State was previously down, and is still down.
'release'  // State was previously down, and now it is up.
'up'       // State was previously up, and it is still up.
```

```
[u,v]  = clientState.point(id,hand)
```

If not hand tracking, or the hand isn't pointing, returns `null`.
If the hand is pointing, returns `[u,v]`, where `-1 <= u <= 1` and `-1 <= v <= 1`.
`u` measures how much the thumb is aiming up or down.
`v` measures how much the index finger is aiming up or down.

Definition of terms for clientState methods:

```
T_or_F -- A true or false value
vec_3  -- An xyz position stored as an array of 3 floats.
mat_16 -- A 4x4 matrix stored as a linear array of 16 floats in column-major order.
hand   -- Valid values: 'left', 'right'
id     -- The unique clientID of an XR client
b      -- Which button is this? Valid values: 0,1,2,3,4,5,6
```

`f` -- Which finger is this? Valid values: 0,1,2,3,4 (or 5 or 6 for the `pinch` method)

g2

The g2 library sits on top of the 2D canvas, providing a virtual canvas that spans `[-1…+1, -1…+1]`.

Example usage

```
import { G2 } from "../util/g2.js";
...
export const init = async model => {
   …
   let g2 = new G2(do_not_animate_flag [, width [, height]]);
   model.txtrSrc(2, g2.getCanvas());
   let obj = model.add('square').txtr(2);
   …
   model.animate(() => {
      g2.clear();
      g2.fillRect(-.5,-.5,1,1);
      …
   });
}
```

If you only want to draw a non-changing image (like fixed labels and diagrams), set the `do_not_animate_flag` to `true`. This will result in much more efficient rendering. Note: `do_not_animate_flag` defaults to `false`, `width` defaults to `1024`, `height` defaults to `width`.

To use a g2 canvas as a texture source:

```
model.txtrSrc(txtr_unit, g2.getCanvas());
```

Note: If the g2 `do_not_animate_flag` is set, then `txtr` is smart enough to optimize performance by not redownloading the canvas image at every animation frame.

Methods of g2:

```
g2.addWidget(obj,type,x,y,color,label,action,size)    // Add a widget
g2.arrow(a,b)            // Draw an arrow from a to b
g2.barChart(x,y,w,h, values,labels, colors)           // Draw a bar chart
g2.clear()               // Clear the canvas
g2.clock(x,y,w,h)        // Draw a clock
g2.computeUVZ(objMatrix) // Get the location and depth of a matrix
g2.drawCurve(n, f)       // Draw a curve using n pts, given f = t => [x,y]
g2.drawOval(x,y,w,h)     // Draw an oval that fits within a rectangle
g2.drawPath(path)        // Draw a path, given [ [x0,y0],[x1,y1],... ]
g2.drawRect(x,y,w,h,r)   // Draw a rectangle, r for rounded corners
g2.drawWidgets(obj)      // Draw the widgets associated with an object
g2.fillCurve(n, f)       // Fill a curve using n pts, given f = t => [x,y]
g2.fillOval(x,y,w,h)     // Fill an oval that fits within a rectangle
g2.fillPath(path)        // Fill a path, given [ [x0,y0],[x1,y1],... ]
g2.fillRect(x,y,w,h,r)   // Fill a rectangle, r for rounded corners
g2.getCanvas()           // Return the actual canvas.
g2.getContext()          // Get the 2D context
```

```
g2.getUVZ(obj)           // Get location+depth of an obj in g2 space
g2.line(a,b)             // Draw a line from a to b
g2.lineWidth(w)          // Set line width
g2.mouseState()          // Get the state of the mouse
g2.noise(true/false)     // Option to add noise for hand-drawn effect
g2.setColor(color,dim)   // Set the drawing color
g2.setFont(f)            // Set the text font
g2.text(text,x,y,alignment,rotation)                  // Draw text
g2.textHeight(h)         // Set text height
g2.update()              // This must be called every animation frame
```

g3

This library allows efficiently display and interaction with 3D dynamic text, procedural diagrams and images, using an API that is similar in spirit to the g2 API .

Usage

```
import { G3 } from "../util/g3.js";

export const init = async model => {
   …
   let g3 = new G3(model, draw => {
      // code to draw one g3 animation frame goes here.
   });
   …
   model.animate(() => {
      g3.update();
   });
}
```

Methods to set properties

```
draw.color(color)              // SET THE DRAWING COLOR
draw.font(font)                // SET THE TEXT FONT
draw.lineWidth(lineWidth)      // SET LINE THICKNESS
draw.textHeight(th)            // SET TEXT HEIGHT
```

Query methods

```
draw.distance(p)               // PERPENDICULAR Z DISTANCE TO A POINT
draw.finger(hand,i)            // POSITION OF USER'S FINGERTIP
draw.pinch(hand,i)             // CHECK FOR A PINCH GESTURE
draw.view()                    // EYE VIEW CURRENTLY BEING RENDERED
```

Methods to draw 3D objects independent of the viewer

```
draw.draw(path)                // DRAW A PATH IN 3D SPACE
draw.fill(path)                // FILL A PATH IN 3D SPACE
draw.line(a,b)                 // DRAW A 3D LINE FROM a TO b
```

Methods to draw objects in 3D that always turn to face the viewer

```
draw.draw2D(path,center)                      // DRAW A 2D PATH
```

```
draw.fill2D(path,center)                        // FILL A 2D PATH
draw.image(image,center,x,y,w,h,sx,sy,sw,sh)    // DRAW AN IMAGE
draw.text(text,center,alignment,x,y,rotation)   // DRAW TEXT
```

NOTE: Methods to set properties or draw objects return `draw`, so they can be chained.

Detailed description of all methods, in alphabetical order

`draw.color(color)` **// SET THE DRAWING COLOR**

These are all valid ways of describing red: `'red'` , `'#ff0000'` , `[1,0,0]`
You can also describe transparent red like this: `'#ff000080'` or `[1,0,0,.5]`

`draw.distance(p)` **// PERPENDICULAR Z DISTANCE TO A POINT**

Returns perpendicular distance along the eye gaze axis from the user's eye to point `p`.

`draw.draw(path)` **// DRAW A PATH IN 3D SPACE**

The `path` argument is in the form: `[ [`x_0`,`y_0`,`z_0`] , [`x_1`,`y_1`,`z_1`] , … ]`

`draw.draw2D(path,center)` **// DRAW A 2D PATH**

The drawing always faces the viewer.
The `path` argument is in the form: `[ [`x_0`,`y_0`] , [`x_1`,`y_1`] , … ]`
In the path, the x and y coordinates are generally in the range `[-1.0 … 1.0]`.
The path is centered at `center`, where `center` is a 3D point in space.

`draw.fill(path)` **// FILL A PATH IN 3D SPACE**

The `path` argument is in the form: `[ [`x_0`,`y_0`,`z_0`] , [`x_1`,`y_1`,`z_1`] , … ]`

`draw.fill2D(path,center)` **// FILL A 2D PATH**

The filled shape always turns to face the viewer.
The `path` argument is in the form: `[ [`x_0`,`y_0`] , [`x_1`,`y_1`] , … ]`
In the path, the x and y coordinates are generally in the range `[-1.0 … 1.0]`.
The path is centered at `center`, where `center` is a 3D point in space.

`draw.finger(hand,i)` **// RETURN POSITION OF A FINGERTIP**

`hand` must be either `'left'` or `'right'`.
`i` must be `0`,`1`,`2`,`3` or `4`, to indicate thumb, index, middle, ring or pinkie, respectively.
If using a controller, `draw.finger(hand)` returns the virtual ping pong ball position.

`draw.font(font)` **// SET THE TEXT FONT**

Valid arguments are `'Helvetica'`, `'Courier'` and `'Times'`.
The default font is Helvetica.

`draw.image(image,center,x,y,w,h,sx,sy,sw,sh)` **// DRAW AN IMAGE**

The displayed image always faces the viewer.

The displayed image is centered at `center`, where `center` is a 3D point in space.
Offset within the 2D image plane by `x` and `y`.
If `width` is positive, set the width of the image to `width`.
If `height` is positive, set the height of the image to `height`.
If either `width` or `height` is omitted, preserve the image's aspect ratio.
Optionally select a rectangle of pixels `sx,sy,sw,sh` as the source sub-image.

```
draw.line(a,b)                  // DRAW A 3D LINE FROM a TO b

draw.lineWidth(lineWidth)       // SET LINE THICKNESS

draw.pinch(hand,i)              // CHECK FOR A PINCH GESTURE
```

`hand` must be either `'left'` or `'right'`.
`i` must be `1`,`2`,`3` or `4`, for thumb touching index, middle, ring or pinkie, respectively,
or else `5` for pointing with your thumb sideways or `6` for pointing with your thumb up.
Returns either `true` or `false`.
If using a controller, `draw.pinch(hand)` returns true if the trigger is pressed.

```
draw.text(text,center,alignment,x,y,rotation) // DRAW TEXT
```

The displayed text always turns to face the viewer.
The displayed text is centered at `center`, where `center` is a 3D point in space.
Multiple lines of text are specified by including `'\n'` within the text string.
Offset within the 2D plane of the text by `x` and `y`.
Optionally set `alignment` to `'left'` or `'right'`. Default alignment is centered.
Optionally specify `rotation`. Each unit represents 90º counterclockwise.

```
draw.textHeight(th)             // SET TEXT HEIGHT

draw.view()                     // EYE VIEW CURRENTLY BEING RENDERED
```

Returns `0` if this is the left eye view, or `1` if this is the right eye view.

Textures

The texturing API consists of a single function:

```
model.setTxtr(src, do_not_animate)
```

The texture `src` can either be a text string, which indicates a texture file, or else an HTML canvas, which indicates a dynamically changing canvas image. If you are on a computer (not a VR/XR headset), then the texture source can be your computer's video camera.

If the texture `src` is a canvas, and `do_not_animate` is set to `true`, then the canvas texture image is only downloaded once to the GPU. For static canvas content (like fixed labels and diagrams) this flag should be set, because it is much more efficient at runtime.

Input System

`inputEvents.js` is located at `js/render/core/inputEvents.js`

`inputEvents` is a global variable accessible by all scenes.

Hand positions
To get a hand position as an `[x,y,z]` vector, use:

```
inputEvents.pos('left')
inputEvents.pos('right')
```

To get hand positions in world space:

```
leftHand.setMatrix (cg.mMultiply(clay.inverseRootMatrix, controllerMatrix.left));
rightHand.setMatrix(cg.mMultiply(clay.inverseRootMatrix, controllerMatrix.right));
```

In the above example, `leftHand` and `rightHand` are objects in the scene.

Head position

To get head matrix of the user, use:

```
cg.mMultiply(clay.inverseRootMatrix, cg.mix(clay.root().inverseViewMatrix(0),
                                            clay.root().inverseViewMatrix(1), .5));
```

The above code takes the average of the left and right eye view matrices. Here is an example of using it:

```
head.setMatrix(cg.mMultiply(clay.inverseRootMatrix,
                            cg.mix(clay.root().inverseViewMatrix(0),
                                   clay.root().inverseViewMatrix(1),.5)));
```

Trigger button status

To get the trigger button status for either hand, use:

```
inputEvents.isPressed('left');
inputEvents.isPressed('right');
```

Events for each hand

```
inputEvents.onMove = hand => {};        // While trigger not pressed
inputEvents.onPress = hand => {};       // On trigger press
inputEvents.onDoublePress = hand => {}; // Both triggers pressed
inputEvents.onClick = hand => {};       // On trigger clicked
inputEvents.onDrag = hand => {};        // While trigger is pressed
inputEvents.onRelease = hand => {};     // On trigger release
```

Shaders

Custom Shader Syntax

You can find an example in `demoShaderNew.js`

To create a custom shader for an object, do:

```
model.customShader(`
<custom vertex shader fields/functions>
```

```
—----------------
<custom vertex shader main>
****************
<custom fragment shader fields/functions>
—----------------
<custom fragment shader main>
`);
```

The `"----------"` line is the separator between declarations and the main function.

The `"*********"` line is the separator between vertex and fragment shaders.

If you only want a custom fragment shader, you may omit the first `"---------"`. (because of compatibility issues, you may also omit the `"**********"` separator.)

If you only want a custom vertex shader, you may omit the second `"---------"`.

Note: the number of `'-'` and `'*'` for each separator does not matter as long as its length is greater than 3.

Custom Shader Example: Morph a sphere by the noise function, then texture the sphere using the noise function

```
let obj = model.add('sphere').flag('myObj');

model.customShader(`
    uniform int myObj;
—----------------
if(myObj == 1) {
    apos.xyz += noise(apos.xyz + uTime * .5) * aNor * .5;
    pos.xyz = obj2Clip(apos.xyz);
}
****************
uniform highp int myObj;
—----------------
if(myObj == 1)
    color *= .5 + noise(1.5 * vAPos);
`);
```

Passing Data into Custom Shaders

You can use `model.setUniform()` to pass data into custom shaders.

Built-In Variables and Functions

Vertex Shader Variables

`float uTime`	- Time since startup.
`mat4 uProj, uView, uModel, uInvModel`	- Matrices.
`vec3 aPos`	- Position in object space.
`vec3 pos`	- Position in clip space.
`vec2 aUV`	- UV in object space.
`vec3 aNor, aTan, aBi`	- Normal, tangent, and bitangent in object space.
`vec3 worldPosition, worldNormal, worldTangent`	- Position, normal, and tangent in world space.

Vertex Shader Functions

```
vec3 obj2World(vec3 p)                          - Transforms a point from object space to world space.
vec3 obj2Clip(vec3 p)                           - Transforms a point from object space to clip space.
vec3 world2Obj(vec3 p)                          - Transforms a point from world space to object space.
float noise(vec3 p)                             - Sample the noise function at p.
```

Fragment Shader Variables

```
float uTime                                     - Time since startup.
mat4 uProj, uView, uModel, uInvModel            - Matrices.
vec3 uViewPosition                              - Camera position in world space.
vec3 vAPos                                      - Position in object space.
vec3 vPos                                       - Position in clip space.
vec3 vNor, vTan, vBi                            - Normal, tangent, and bitangent in world space.
vec2 vUV                                        - UV in object space.
vec3 worldPosition, worldNormal, worldTangent   - World space position, normal, tangent.
float uOpacity                                  - Opacity value used in alpha blending.
```

Fragment Shader Functions

```
vec3 obj2World(vec3 p)                          - Transforms a point from object space to world space.
vec3 world2Obj(vec3 p)                          - Transforms a point from world space to object space.
float noise(vec3 p)                             - Sample the noise function at p.
```

Audio APIs

Basic Spatial Audio

See `/js/scenes/demoSoundWrapper.js` for a demo.

```
createSoundSource(soundIndex, soundUrl, initialPosition, loop, volume);
```

- `soundIndex` *(number)*: Unique id for the sound source. An integer starting from 0.
- `soundUrl` *(string)*: The relative or absolute URL path to the sound file.
- `initialPosition` *(array)*: Position vector `[x,y,z]` defining source's initial location.
- `loop` *(boolean)*: Should sound loop? `true` for looping, `false` for single playback.
- `volume` *(number)*: Volume level as a percentage, where `1.0` represents 100% volume.

```
Object = model.addAudio(soundIndex)
```

- Associates a sound source (by soundIndex) with the 3D object.

```
Object.playAudio()
```

- Starts audio playback for the associated sound source.

```
Object.stopAudio()
```

- Stops the audio playback.

Acoustic Properties Setup

See `/js/util/spatial-audio.js`.

`roomDimensions`

- The `roomDimensions` object specifies the size of the room in meters. This includes the width, height, and depth of the virtual space.

```
let roomDimensions = {
  width:  6,      // Width of the room in meters
  height: 2.5,    // Height of the room in meters
  depth:  6       // Depth of the room in meters
};
```

`roomMaterials`

- The `roomMaterials` object defines the material properties of each surface in the room. These properties influence how sound reflects and absorbs on the respective surfaces, simulating realistic acoustics.

```
let roomMaterials = {
  left: 'curtain-heavy',                  // Left wall material
  right: 'curtain-heavy',                 // Right wall material
  front: 'curtain-heavy',                 // Front wall material
  back: 'curtain-heavy',                  // Back wall material
  down: 'polished-concrete-or-tile',      // Floor material
  up: 'wood-ceiling'                      // Ceiling material
};
```